

1. Arquitectura del Procesador STX4

1.1. Modelo de Registros

La CPU del sistema RTM32, llamada STX4, es un procesador RISC de 32 bits ortogonal que cuenta con un banco de 32 registros de propósito general para el usuario numerados de 0 a 31 como se observa en la tabla 1.1 donde se exhibe la designación usada por el ensamblador (*mnemonic*) para cada uno y el propósito o convención que deberá seguir el programador y/o la herramienta de desarrollo que haga uso de los mismos.

Reg.	Mnemonic	Propósito Convención
\$0	\$zero	Hardwired a cero ¹
\$1	\$ra	Dirección de retorno
\$2,...,\$7	\$t0,...,\$t5	Registros temporarios de propósito general
\$8,\$9	\$k0,\$k1	Reservados para el sistema
\$10,...,\$13	\$a0,...,\$a3	Registros para paso de argumentos de función
\$14,\$15	\$v0,\$v1	Registros de retorno de valor de una función
\$16,...,\$19	\$t6,...,\$t9	Registros temporarios de propósito general
\$20,...,\$27	\$s0,...,\$s7	Registros almacenables de propósito general
\$28	\$fp	Registro puntero de marco
\$29	\$gp	Registro puntero global
\$30	\$sp	Registro puntero de pila
\$31	\$at	Registro temporario de uso habitual para el ensamblador

Tabla 1.1: Uso y convención de los registros RTM32

El registro cero (\$zero o \$0) siempre contiene el valor 0. Está cableado en el hardware, así que no se puede modificar, aunque puede usarse como destino en cualquier instrucción.

El registro \$at (*Assembler Temporary*) se usa para valores temporales dentro de pseudo-instrucciones. No se preserva entre function calls. Por ejemplo, con el comando (`slt $at, $a0, $s2`), \$at se setea en 1 si \$a0 es menor que \$s2; de lo contrario, se setea en 0.

Los registros \$v0,\$v1 se usan para retornar valores desde funciones. No se preservan entre subrutinas (*function calls*). Normalmente solo se usa \$v0 pero es posible retornar valores de 64 bits mediante \$v1 (MSB). Dado que esto último no es usual este registro puede usarse también como un temporal con el alias \$t10.

Los registros \$a0,...,\$a3 (*argument*) se usan para pasar argumentos a funciones. No se preservan entre function calls al igual que los registros temporales \$t0,...,\$t9 (*temporaries*) que son usados por el assembler o por el programador de assembly para almacenar valores intermedios. Los registros \$s0,...,\$s7 (*Saved Temporary*) se usan para almacenar valores de mayor duración y sí se preservan entre function calls. Los registros \$k0,\$k1 están reservados para uso del kernel del sistema operativo. Pueden cambiar de forma impredecible en cualquier momento porque son utilizados por los interrupt handlers.

Los registros de puntero (*pointer registers*) son una familia de registros que contiene direcciones de memoria, a lo que se les asigna funciones muy particulares. Todos ellos se preservan entre function calls y no todos son de uso obligatorio, por lo que pueden cumplir una función secundaria con otro alias. Por ejemplo \$fp suele ser \$s8 y \$gp es \$s9. Es importante entender

que mientras se los use como otros registros no podrán ser punteros y viceversa.

A continuación detallamos la función asignada a cada uno de ellos:

- Puntero global **\$gp** (*Global Pointer*): normalmente guarda un puntero al área de datos globales (para que pueda accederse usando memory offset addressing).
- Puntero de pila **\$sp** (*Stack Pointer*): se usa para almacenar la dirección a donde apunta la pila actualmente.
- Puntero de marco **\$fp** (*Frame Pointer*): se usa para almacenar la dirección base del marco de pila (*Stack Frame*).
- Dirección de retorno **\$ra** (*Return Address*): almacena la dirección de retorno (la ubicación del programa a la que una función debe volver). Este puntero es usado comúnmente para las instrucciones JAL y JALR.

Todos los registros mencionados son utilizados regularmente en la CPU y accesible a través de las instrucciones pero hay otros registros como el contador de programa **\$pc** (*program counter*) que es manipulado exclusivamente por la CPU e indirectamente por las instrucciones de cambio de flujo. Los registros de funciones especiales (*special function registers*) permiten inspeccionar o setear distintas condiciones de la CPU y se acceden en forma indirecta mediante instrucciones específicas como CFT y CTS. La CPU STX4 tiene 5 registros especiales que detallamos a continuación:

- Palabra de estado del programa **\$psw** (*Program Status Word*): almacena el estado de ejecución del procesador, incluyendo las banderas aritméticas, el nivel de privilegio y el estado global de las interrupciones.
- Registro de causa de la excepción **\$ecr** (*Exception Cause Register*): contiene el código que identifica la excepción o interrupción que provocó la transferencia de control al manejador correspondiente.
- Contador de programa de la excepción **\$epc** (*Exception Program Counter*): almacena la dirección de la instrucción cuya ejecución fue interrumpida por la excepción o interrupción, permitiendo reanudar la ejecución cuando corresponda.
- Dirección virtual incorrecta **\$bva** (*Bad Virtual Address*): almacena la dirección virtual que originó una excepción de acceso a memoria.
- Registro base vectorizado **\$vbr** (*Vector Base Register*): contiene la dirección base de la tabla de vectores de excepción e interrupción. Cuando se resetea el procesador este registro se carga con el valor 0xF0000000

1.2. Formatos de Instrucción

Las instrucciones constan de un ancho fijo de 32 bits y se decodifican mediante campos bit a bit estructurados en cuatro tipos llamados: R, I, L y J. Todas las instrucciones comparten el mismo campo llamado **opcode** o **código de operación** (*operation code*) formado por los primeros 5 bits más significativos (bit 27 al 31). El valor del opcode identifica al tipo de instrucción como se muestra en la tabla A.1. No todos los opcodes son válidos, por ejemplo 0x22 no lo es y por lo tanto cualquier instrucción que utilice dichos opcodes son instrucciones inválidas y el uso de los mismas genera excepciones.

El formato tipo R (*register*) que se muestra en la figura 1.1 se utiliza principalmente para operaciones aritméticas y lógicas entre registros, así como otra variedad de operaciones como desplazamientos, movimientos en la memoria y tareas más específicas.

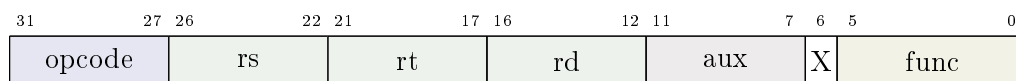


Figura 1.1: Formato tipo R

Cómo todas comparten el mismo opcode, la CPU las diferencia por el campo **func** o función (*function*) que se encuentra en los 6 bits menos significativos (bit 0 al 5). En la tabla A.2 se provee una lista de las instrucciones tipo R y sus valores de función asociados.

Los campos **rs**, **rt** y **rd** especifican registros (de allí el nombre de tipo R), haciendo referencia al **registro fuente** (*source register*), el **registro temporal** o secundario (*temporal register*) y el **registro destino** (*destiny register*), pero el campo que está entre los bits 8 y 12 es más complejo de explicar ya que cumple dos funciones y por eso se le llama **aux**. En algunas operaciones constituye una referencia **auxiliar** pero en otras es una **constante inmediata** de 5 bits que puede tomar valores entre 0 y 31. Los detalles sobre su uso en las operaciones que sean afectadas se discutirá más adelante según la operación.

El formato tipo I o **inmediato** (*immediate*) se utiliza para las operaciones que requieren del uso de una constante **imm** para resolver la instrucción. El nombre proviene del modo de direccionamiento llamado igual, ya que el operando está inmediatamente anexo a la propia instrucción. Su estructura se muestra en la figura 1.2 y se compone de 4 campos. En este formato el campo **opcode** determina exactamente cuál es la instrucción.

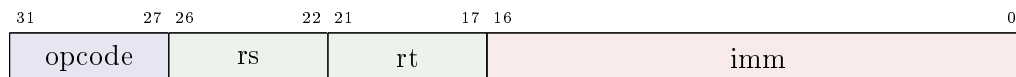


Figura 1.2: Formato tipo I

Este tipo de instrucción se utiliza para movimientos de carga o descarga en la memoria donde es necesario calcular una **dirección efectiva EA** (effective address) dependiendo de un offset (**imm**). El campo **rs** siempre especifica el registro que se va a cargar o descargar, mientras que **rt** es un apuntador (pointer) y el campo **imm** almacena una constante inmediata de 17 bits en complemento a dos llamada desplazamiento (*offset*). La suma del puntero y su desplazamiento conforman la **dirección efectiva EA** y a este modo de direccionamiento se lo conoce como modo indexado. Esta es la dirección de memoria donde se encuentra el dato a traer a la CPU o donde la CPU lo va a descargar.

ADVERTENCIA

Algunas lecturas y escrituras de memoria realizan verificación de alineación (alignment checking). Una dirección EA no alineada utilizada por una instrucción que necesita operar con palabras, como por ejemplo LW generará una excepción deteniendo el procesador si EA no es múltiplo de 4.

Este formato también se utiliza para operaciones de salto condicional (*branch*). Según el tipo de operación **BEQ**, **BNE**, etc, luego de analizar la condición, si la misma es verdadera entonces la CPU realizará el salto correspondiente cambiando el valor del PC a uno nuevo cuyo cálculo se explica a continuación. La clave para entender **imm** es que no almacena una

dirección de salto sino un offset en palabras que permite tener un desplazamiento de $\approx \pm 2^{16}$ hacia adelante o atras debido a que está almacenado en complemento a dos. La dirección se salto responde a la fórmula:

$$PC = PC + 4 * imm$$

La constante 4 que se traduce en el sentido que no es necesario guardar los dos últimos bits de la dirección es porque este procesador al ser regular no admite que ninguna instrucción no esté alineada (direcciones múltiplos de 4).

Por último, tenemos las instrucciones SLTI y SLTIU han quedado como un rezago histórico que permite testear la condición menor que con y sin signo respecto de una constante. Permiten algunas comparaciones elementales sin necesidad de tener que tener todos los datos previamente en registros, lo cuál puede ser muy conveniente, así como el caso de ADDI que es extremadamente útil para incrementar o decrementar contadores.

El formato L es una variante del formato I. Existe porque es absurdo utilizar una constante de 17 bits para operaciones lógicas como ANDI, ORI y XORI. Por lo tanto básicamente se comprimió `imm` a 16 bits quedando un bit `h` disponible para indicar si dicha constante opera en la parte inferior de la palabra (`h = 0`) o superior (`h = 1`) de la palabra².

Solo queda mencionar LUI que carga la constante `imm` en los 16 bits MSB del registro especificado por `rt` dejando los restantes en 0s. Esta instrucción es vital para cargar constantes arbitrarias de 32 bits en registros mediante la combinación con ORI.

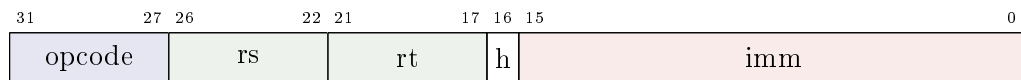


Figura 1.3: Formato tipo L

Finalmente arribamos al último formato que se muestra en la figura 1.4. Las instrucciones tipo J, se utilizan para los saltos incondicionales J y JAL con sus respectivos opcodes 0x2 y 0x3. Básicamente poseen dos campos, el `opcode` para identificarlas y la dirección de salto `jump address` en palabras de memoria no signadas (*unsigned*). Esto significa al igual que en los branch que este valor debe almacenarse omitiendo los últimos dos bits de la dirección a saltar.

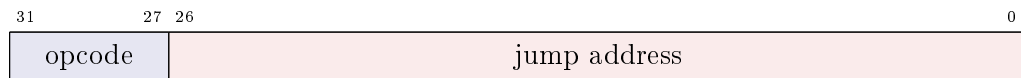


Figura 1.4: Formato tipo J

La única particularidad relevante dado que el campo `jump address` es de 27 bits la dirección almacenada representa 29 bits, quedan 3 bits faltantes para obtener la dirección final de 32 bits. Estos 3 bits se extraen del los 3 MSB del PC (`PC[31:29]`) y se concatenan con los 27 bits de `jump address` más dos bits que son siempre 0s.

²Actualmente hay un bug importante a solucionar en la instrucción ANDI

A. Instruction Set Reference

Opcode	Type	Mnemo	Operands	Operation
00000	R			see table A.2
00001	I	ADDI	rs rt imm	$R[rt] = R[rs] + SE(imm)$ ⁽¹⁾
00010	J	J	address	$PC = E(address)$ ⁽²⁾
00011	J	JAL	address	$R[31] = PC + 4; PC = E(address)$
00100	L	ANDI/H	rs rt ims	$R[rt] = R[rs] \& ZE(ims)/ZC(ims)$ ^(3,4)
00101	L	ORI/H	rs rt ims	$R[rt] = R[rs] ZE(ims)/ZC(ims)$
00110	L	XORI/H	rs rt ims	$R[rt] = R[rs] \oplus ZE(ims)/ZC(ims)$
00111	L	LUI	rt ims	$R[rt] = ZC(ims)$
01000	I	LW	rs rt imm	$R[rt] = M[EA(rs, imm)]$ ⁽⁵⁾
01001	I	SW	rs rt imm	$M[EA(rs, imm)] = R[rt]$
01010	I	SH	rs rt imm	$M[EA(rs, imm)][15:0] = R[rt][15:0]$
01011	I	SB	rs rt imm	$M[EA(rs, imm)][7:0] = R[rt][7:0]$
01100	I	LH	rs rt imm	$R[rt] = SHE(M[EA(rs, imm)][15:0])$ ⁽⁶⁾
01101	I	LHU	rs rt imm	$R[rt] = ZE(M[addr][15:0])$
01110	I	LB	rs rt imm	$R[rt] = \{24\{M[addr](7:0)[7]\}, M[addr](7:0)\}$ ⁽⁷⁾
01111	I	LBU	rs rt imm	$R[rt] = \{24'b0, M[addr](7:0)\}$ ⁽⁸⁾
10000	I	BEQ	rs rt imm	if $(R[rs] == R[rt])$ $PC = PC + 4 + Branch(imm)$ ⁽⁹⁾
10001	I	BNE	rs rt imm	if $(R[rs] != R[rt])$ $PC = PC + 4 + Branch(imm)$
10010	I	BLT	rs rt imm	if $(R[rs] < R[rt])$ $PC = PC + 4 + Branch(imm)$
10011	I	BGT	rs rt imm	if $(R[rs] > R[rt])$ $PC = PC + 4 + Branch(imm)$
10100	I	BLE	rs rt imm	if $(R[rs] <= R[rt])$ $PC = PC + 4 + Branch(imm)$
10101	I	BGE	rs rt imm	if $(R[rs] >= R[rt])$ $PC = PC + 4 + Branch(imm)$
10110	I	SLTI	rs rt imm	$R[rt] = (R[rs] < SE(imm)) ? 1 : 0$
10111	I	SLTIU	rs rt imm	$R[rt] = (R[rs] < ZE(imm)) ? 1 : 0$

Tabla A.1: RTM32 Instruction Set sorted by opcode

- (1) $SE(imm) = \{15'imm[16], imm\}$ extiende **imm** con signo
- (2) $E(address) = \{PC+4[31:29], address, 2'b0\}$ extiende **address**
- (3) $ZE(ims) = \{16'b0, imm\}$ extiende **ims** con 0s
- (4) $ZC(ims) = \{imm, 16'b0\}$ concatena **ims** con 0s
- (5) $EA(rs, imm) = R[rs] + SE(imm)$ calcula la dirección efectiva **EA**
- (6) $SHE(h) = \{16'\{h[15]\}, h\}$ extiende **h** (media palabra) usando signo
- (7) $SBE(byte) = \{24'\{byte[7]\}, byte\}$ extiende **byte** usando signo
- (8) $ZBE(byte) = \{24'b0, byte\}$ extiende **byte** usando 0s
- (9) $Branch(imm) = \{13'\{imm[16]\}, imm, 2'b0\}$ calcula el offset de salto

Func	Mnemo	Operands	Operation
000000	SLL	rs rt rd aux	$R[rd] = R[rt] \ll aux$
000001	SRL	rs rt rd aux	$R[rd] = R[rt] \gg aux$
000010	SRA	rs rt rd aux	$R[rd] = R[rs] \ggg aux$
000011	SLLR	rs rt rd	$R[rd] = R[rt] \ll R[rs][4:0]$
000100	SRLR	rs rt rd	$R[rd] = R[rt] \gg R[rs][4:0]$
000101	SRAR	rs rt rd	$R[rd] = R[rt] \ggg R[rs][4:0]$
000110	CFS	rs aux	$R[rs] = S[aux]$
000111	CTS	rs aux	$S[aux] = R[rs]$
001000	AND	rs rt rd	$R[rd] = R[rs] \& R[rt]$
001001	OR	rs rt rd	$R[rd] = R[rs] R[rt]$
001010	XOR	rs rt rd	$R[rd] = R[rs] \oplus R[rt]$
001011	NOR	rs rt rd	$R[rd] = \neg (R[rs] R[rt])$
001100	SLT	rs rt rd	$R[rd] = (R[rs] < R[rt]) ? 1 : 0$
001101	SLTU	rs rt rd	$R[rd] = (R[rs] < R[rt]) ? 1 : 0^{(8)}$
001110	JR	rs	$PC = R[rs]$
001111	JALR	rs rt	$R[31] = PC + 4; PC = R[rs]$
010000	LHX	rs rt rd	$R[rt] = \{16\{M[R[rs] + R[rd]](15:0)[15], M[R[rs] + R[rd]](15:0)\}$
010001	LHUX	rs rt rd	$R[rt] = \{16'b0, M[R[rs] + R[rd]](15:0)\}$
010010	LBX	rs rt rd	$\{24\{M[R[rs] + R[rd]](7:0)[7], M[R[rs] + R[rd]](7:0)\}$
010011	LBUX	rs rt rd	$R[rt] = \{24'b0, M[R[rs] + R[rd]](7:0)\}$
010100	LWX	rs rt rd	$R[rt] = M[R[rs] + R[rd]]$
010101	MUL	rs rt rd	$R[rd] = \{R[rs] \times R[rt]\}(31:0)$
010110	MULH	rs rt rd	$R[rd] = \{R[rs] \times R[rt]\}(63:32)$
010111	MULHU	rs rt rd	$R[rd] = \{R[rs] \times R[rt]\}(63:32)^{(8)}$
011000	DIV	rs rt rd	$R[rd] = R[rs] / R[rt]$
011001	DIVU	rs rt rd	$R[rd] = R[rs] / R[rt]^{(8)}$
011010	REST	rs rt rd	$R[rd] = R[rs] \% R[rt]$
011011	RESTU	rs rt rd	$R[rd] = R[rs] \% R[rt]^{(8)}$
011100	ADD	rs rt rd	$R[rd] = R[rs] + R[rt]$
011101	SUB	rs rt rd	$R[rd] = R[rs] - R[rt]$
100000	TRAP	aux	$EPC = PC + 4; PC = M[aux \ll 2]$
100001	RFT		$PC = EPC$

Tabla A.2: R-type instructions sorted by func